

Javascript

Function expression, arrow, objects , closure, hoisting

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h1>Function</h1>

    <script>
      // function declaration

      // function expression
      let func = function display() {
        console.log("Display function ");
      };

      // display();
      console.log(func);
      func();

      //arrow function
      // Es6+
      let arrow = (a, b) => {
        console.log("Fiit function", a, b);
      };
      console.log(arrow);
      arrow(10, 20);

      // IIFE => immediate invoked function
      //anonymous function
      function data() {}

      data();

      (function (a, b) {
        console.log("Immediate invovked function ", a, b);
      })(10, 20);

      // closure => (lexical scope)
      // inner function remember outer function variable

      function outer() {
```

```

    let count = 10;

    return function inner() {
        count++;
        console.log(count);
    };
}

let result = outer();

console.log(result);

result();
result();
result();

// callback function

function greet(name, sayHello) {
    console.log(name);
    sayHello(thirdFunc);
}

function sayHello(callback2) {
    console.log("Welcome");
    callback2();
}

function thirdFunc() {
    console.log("third callback function ");
}

greet("Fiit", sayHello);

// hoisting

hoist();

function hoist() {
    console.log("hoisting");
}

console.log(hoistVar);

var hoistVar = 10;

// console.log(hoistLet);

```

```
let hoistLet = 20;

// top of the scope
// var hoistvar ;
// let hoistLet; => tdz
// function hoist(){
//     console.log("hoisting");

// }

// hoist();

// function hoist(){
//     console.log("hoisting");

// }

// console.log(hoistVar);

// var hoistVar = 10;

// console.log(hoistLet);

// let hoistLet = 20;

// objects => key:value pairs

let person = {
  name: "Fiit",
  course: "Python full stack",
  address: {
    street: "vijayaragavapuram",
    landmark: {
      location: "Aavin milk factory",
    },
  },
  "roll no": 12,
  greet() {
    console.log("Greet message from person");
  },
};

let obj2 = {
  duration: 6,
};

console.log(Object.keys(person));
```

```

    console.log(Object.values(person));

    console.log(Object.entries(person));

    let merged = Object.assign(person, obj2);

    console.log(merged);

    console.log(typeof person);
    person.greet();

    person["roll no"] = 50;
    console.log(person["roll no"]);

    console.log(person);

    person.name = "Harini";
    console.log(person.name);

    console.log(person.course);

    console.log(person.address.street);

    person.address.landmark.location = "Vellore";
    console.log(person.address.landmark.location);
</script>
</body>
</html>

```

Theory

1. Function Declaration vs Expression

Definition:

- **Declaration** → Named function defined with function keyword.
- **Expression** → Function assigned to a variable.
- **Key point:** Declarations are hoisted, expressions are not.

Syntax:

function add(a, b) { return a+b; } // declaration

let func = function(a, b) { return a+b; }; // expression

2. Arrow Functions

Definition: Introduced in ES6, arrow functions provide a shorter syntax. **Use case:** Useful for callbacks and concise code. **Key point:** Do not bind their own this.

Syntax:

```
let arrow = (a, b) => a+b;
```

3. IIFE (Immediately Invoked Function Expression)

Definition: A function that runs immediately after being defined. **Use case:** Avoids polluting global scope.

Syntax:

```
(function(a, b){ console.log(a+b); })(10, 20);
```

4. Closures

Definition: A closure is a function that remembers variables from its lexical scope even after the outer function has finished executing. **Use case:** Data privacy, function factories, maintaining state.

Syntax:

```
function outer(){  
  let count = 0;  
  return function inner(){ count++; return count; };  
}  
  
let result = outer();  
result(); // 1  
result(); // 2
```

5. Callback Functions

Definition: A function passed as an argument to another function. **Use case:** Asynchronous programming, event handling.

Syntax:

```
function greet(name, callback){ callback(name); }
```

6. Hoisting

Definition: JavaScript moves function declarations and var declarations to the top of scope during compilation.

Key point:

- var → hoisted but initialized as undefined.
- let/const → hoisted but in Temporal Dead Zone (TDZ).
- Functions → fully hoisted.

7. Objects

Definition: Objects store data as key-value pairs. **Use case:** Represent entities with properties and methods. **Key point:** Can nest objects and use methods like Object.keys(), Object.values(), Object.assign().

Syntax:

```
let person = { name: "Fiit", course: "Python", greet(){ console.log("Hello"); } };
```

Coding Round Questions

- Function types → Explain difference between function declaration and expression with examples.
- Arrow function → Write an arrow function to multiply two numbers.
- IIFE → Demonstrate an IIFE that prints “Hello World”.
- Closure → Write a closure that maintains a counter.
- Callback → Demonstrate a callback function that prints a greeting.
- Hoisting → Explain hoisting with var, let, and function.
- Objects → Create an object and update its properties dynamically.

Machine Coding Round Challenge

Problem: Task Manager Application

Requirements:

- Create a TaskManager object with:
 - Property tasks (array).
 - Method addTask(task) → adds a task.

- Method `listTasks()` → lists all tasks.
 - Method `removeTask(index)` → removes a task by index.
- Use **closures** to maintain a private counter of tasks added.
- Demonstrate adding, listing, and removing tasks.